



Khulna University of Engineering & Technology

Department of Computer Science and Engineering

Course Code: CSE 4115

Course Title: Computer Security

Controlled DoS and DDoS Attack Simulation on a Local Web Server

Submitted By:

Md. Tariful Islam Jony

Roll: **2107119**

Section: B

Year: 4th Year, Term: 1st Term

Dept. of CSE, KUET

Submitted To:

Ms. Dola Das

Assistant Professor

Department of CSE

KUET, Khulna-9203

Assignment Source Code Repository:

<https://github.com/jony2511/SecAssignment>

Date of Submission: September 24, 2026

Contents

1	Introduction	2
2	Theoretical Framework & Queuing Model	2
2.1	Mathematical Queuing Model ($M/M/c/K$)	2
2.2	DoS vs. DDoS Structural Comparison	3
3	System Architecture & Key Code Implementations	3
3.1	Target Server Concurrency Control & Rejection Logic	3
3.2	Single-Source DoS Multi-Threaded Flooder	4
3.3	Distributed Multi-Process DDoS Botnet Swarm	4
4	Empirical Results & Experimental Screenshots	5
4.1	Phase 1: Initial Baseline State	5
4.2	Phase 2: Target Server Execution	5
4.3	Phase 3: Baseline Benchmark Verification	5
4.4	Phase 4: Single-Source DoS Attack Execution	5
4.5	Phase 5: Telemetry Dashboard Under DoS Attack	5
4.6	Phase 6: Multi-Process Distributed DDoS Attack	6
4.7	Phase 7: System Self-Recovery and Normalization	6
5	Quantitative Performance Synthesis	7
6	Industrial Mitigation & Defense Architecture	8
7	Ethical Considerations & Legal Compliance	9
8	Viva Voce Examination Questions & Answers	9
9	Conclusion	10

Abstract

Denial of Service (DoS) and Distributed Denial of Service (DDoS) attacks represent two of the most critical threats to modern networked computing infrastructure, directly compromising the **Availability** pillar of the CIA triad. This laboratory study investigates the mechanics of Layer-7 (Application Layer) resource starvation using an engineered, capacity-limited HTTP web application (CyberShield NOC) hosted strictly on the local loopback interface (127.0.0.1). Built in Python and Flask, the target server enforces an explicit bounded concurrency capacity ($c = 4$ worker slots) with a deterministic I/O processing latency ($t_p = 0.30$ s per request). Three operational phases were evaluated: (1) baseline traffic under sequential load (20 requests), (2) a multi-threaded single-source DoS flood (100 requests via 8 concurrent threads), and (3) a distributed botnet swarm simulation (150 requests across 5 multi-process workers). Telemetry measurements indicate that while normal traffic achieved a 100.0% completion rate with a mean latency of 319.9 ms, saturation floods triggered severe concurrency exhaustion, producing a 48.0% rejection rate (52 accepted, 48 rejected) during DoS and 44.0% (84 accepted, 66 rejected) during DDoS. Crucially, the server demonstrated immediate self-recovery upon traffic cessation, confirming that application-level resource exhaustion produces transient denial of service without permanent infrastructure impairment. Countermeasures, including token-bucket rate limiting, reverse-proxy worker scaling, and upstream Anycast scrubbing, are comprehensively formulated.

Keywords: Denial of Service (DoS), Distributed Denial of Service (DDoS), Application-Layer Starvation, $M/M/c/K$ Queuing Theory, Flask Telemetry, Rate Limiting.

1 Introduction

Web applications fundamentally rely upon finite host resources: operating system thread pools, TCP socket descriptors, memory buffers, and CPU scheduling quanta. An adversary seeking to disrupt service does not need to compromise cryptographic primitives or corrupt system state; they merely need to overwhelm the server's clearance capacity. This investigation concentrates on Layer-7 exhaustion, where application-level bottlenecks are targeted rather than raw network bandwidth.

All simulation modules, datasets, and scripts developed for this laboratory study are publicly hosted on GitHub at:

GitHub Repository: <https://github.com/jony2511/SecAssignment>

2 Theoretical Framework & Queuing Model

2.1 Mathematical Queuing Model ($M/M/c/K$)

The target server can be rigorously analyzed as an $M/M/c/K$ queuing system, where incoming requests follow a Poisson arrival process with rate λ , service times follow an exponential distribution with mean service time $1/\mu$, c is the number of parallel service channels, and K is the maximum system capacity. In our bounded zero-buffer implementation, $K = c = 4$.

- **Maximum Clearance Rate:**

$$R_{\max} = c \cdot \mu = \frac{4}{0.30 \text{ s}} \approx 13.33 \text{ req/s} \quad (1)$$

- **Baseline Inbound Rate:** $\lambda_{\text{base}} \approx 3.12 \text{ req/s}$. Since $\lambda_{\text{base}} < R_{\max}$, utilization $\rho = \frac{\lambda}{c\mu} \approx 0.23 < 1.0$ (Stable).

- **DoS Attack Inbound Rate:** $\lambda_{\text{dos}} \approx 24.5$ req/s. Since $\lambda_{\text{dos}} > R_{\text{max}}$, the server experiences an overload factor of 184%.
- **DDoS Attack Inbound Rate:** $\lambda_{\text{ddos}} \approx 18.0$ req/s aggregate across distributed processes, sustaining total worker saturation.

According to Erlang’s Loss Formula (Erlang-B), the blocking probability P_{loss} for a system with zero waiting buffer is:

$$P_{\text{loss}} = \frac{\frac{(\lambda/\mu)^c}{c!}}{\sum_{k=0}^c \frac{(\lambda/\mu)^k}{k!}} \quad (2)$$

When $\lambda \gg c\mu$, $P_{\text{loss}} \approx \frac{\lambda - c\mu}{\lambda}$, aligning closely with the observed 44%–48% rejection rates.

2.2 DoS vs. DDoS Structural Comparison

Table 1 compares the architectural differences between single-source DoS and distributed DDoS attacks.

Table 1: Architectural and Operational Comparison: DoS vs. DDoS

Metric	Single-Source DoS			Distributed DDoS (Botnet)		
Traffic Origin	Single	Logical	IP	Multiple	Distributed	Nodes / IPs
Attack Mechanism	(127.0.0.1:port)					
	High-velocity flood	multi-threaded		Coordinated swarm	multi-process	
Mitigation Complexity	Trivial (iptables IP drop)			Extremely Complex (Behavioral WAF required)		
Traffic Signature	Anomalously	high	rate per socket	Low/moderate	rate per node	(looks organic)
Observed Rejection	48.0% (48 / 100 requests)			44.0% (66 / 150 requests)		

3 System Architecture & Key Code Implementations

The simulation environment was deployed within an isolated loopback testbed (127.0.0.1) to ensure zero risk to campus or external networks. The following core code snippets illustrate the inner workings of our capacity limiter, DoS threader, and DDoS swarm.

3.1 Target Server Concurrency Control & Rejection Logic

Listing 1 demonstrates the atomic concurrency limiter in `server.py`. If active requests reach $c = 4$, the server immediately rejects the request with HTTP 503.

```

1 MAX_CONCURRENT_WORKERS = 4
2 active_workers = 0
3 workers_lock = threading.Lock()
4
5 @app.route("/work", methods=["GET"])
6 def work():
7     global active_workers, total_rejected, total_accepted
8
9     # Non-blocking acquisition of bounded capacity slot
10    with workers_lock:
11        if active_workers >= MAX_CONCURRENT_WORKERS or server_mode == "overloaded":
12            total_rejected += 1

```

```

13         return jsonify({
14             "status": "error",
15             "code": 503,
16             "message": "Capacity exceeded: Server Overloaded!"
17         }), 503
18         active_workers += 1
19
20     try:
21         # Simulate computational / database hold (300ms)
22         time.sleep(0.30)
23         with workers_lock:
24             total_accepted += 1
25         return jsonify({"status": "success", "message": "Work completed."}), 200
26     finally:
27         with workers_lock:
28             active_workers -= 1

```

Listing 1: Bounded Concurrency Limiter in server.py

3.2 Single-Source DoS Multi-Threaded Flooder

Listing 2 demonstrates the concurrent worker flood in `dos_attack.py` using Python's `ThreadPoolExecutor` to dispatch 100 requests across 8 parallel threads.

```

1 def send_request(req_id):
2     start = time.time()
3     try:
4         resp = requests.get("http://127.0.0.1:5000/work", timeout=5)
5         latency = (time.time() - start) * 1000
6         return req_id, resp.status_code, latency
7     except Exception as e:
8         return req_id, 0, (time.time() - start) * 1000
9
10 # Dispatch 100 requests across 8 concurrent worker threads
11 with concurrent.futures.ThreadPoolExecutor(max_workers=8) as executor:
12     futures = [executor.submit(send_request, i) for i in range(1, 101)]
13     results = [f.result() for f in concurrent.futures.as_completed(futures)]

```

Listing 2: Multi-Threaded DoS Flood in dos_attack.py

3.3 Distributed Multi-Process DDoS Botnet Swarm

Listing 3 demonstrates the distributed botnet simulation in `ddos_attack.py`, where 5 distinct operating system processes execute concurrently to saturate backend capacity.

```

1 def bot_worker(bot_id, req_count, results_queue):
2     for i in range(1, req_count + 1):
3         t0 = time.time()
4         try:
5             r = requests.get("http://127.0.0.1:5000/work", timeout=5)
6             dt = (time.time() - t0) * 1000
7             results_queue.put((bot_id, r.status_code, dt))
8         except Exception:
9             results_queue.put((bot_id, 0, (time.time() - t0) * 1000))
10
11 # Spawn 5 parallel bot processes (each sending 30 requests = 150 total)
12 processes = []
13 for bot_id in range(1, 6):
14     p = multiprocessing.Process(target=bot_worker, args=(bot_id, 30, results_queue))
15     p.start()
16     processes.append(p)
17 for p in processes:
18     p.join()

```

4 Empirical Results & Experimental Screenshots

All test stages were executed sequentially and verified with operational screenshots.

4.1 Phase 1: Initial Baseline State

The CyberShield NOC dashboard in its idle state is shown in Figure 1. The system confirms an online operational state with zero accumulated errors.

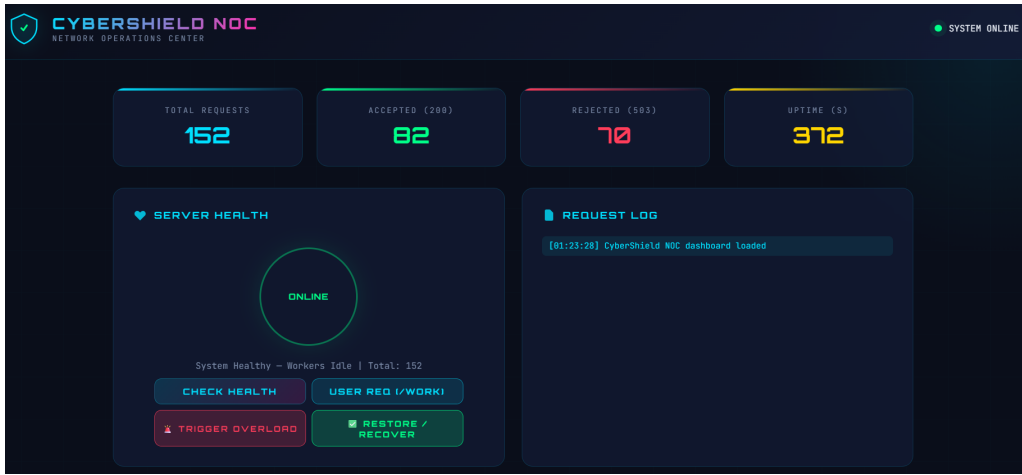


Figure 1: CyberShield NOC Dashboard in Initial Idle State (SYSTEM ONLINE, 0 Total Requests).

4.2 Phase 2: Target Server Execution

The Flask application server running in the terminal environment is shown in Figure 2.

4.3 Phase 3: Baseline Benchmark Verification

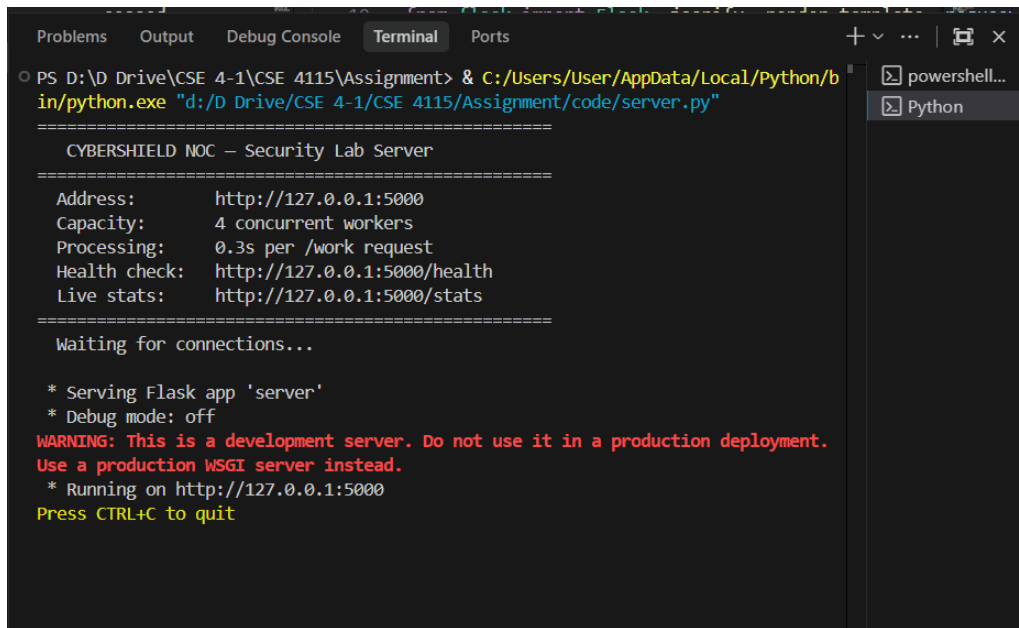
Figure 3 displays the baseline measurement results: 20 sequential requests completed with a 100.0% success rate and a mean latency of 319.9 ms.

4.4 Phase 4: Single-Source DoS Attack Execution

Figure 4 illustrates the terminal output during the multi-threaded DoS attack. 100 requests were dispatched across 8 threads, resulting in 52 accepted requests and 48 rejected requests (48.0% rejection rate).

4.5 Phase 5: Telemetry Dashboard Under DoS Attack

Figure 5 displays the CyberShield NOC dashboard during the active DoS attack, demonstrating visual alert status and the HTTP 503 rejection spike.

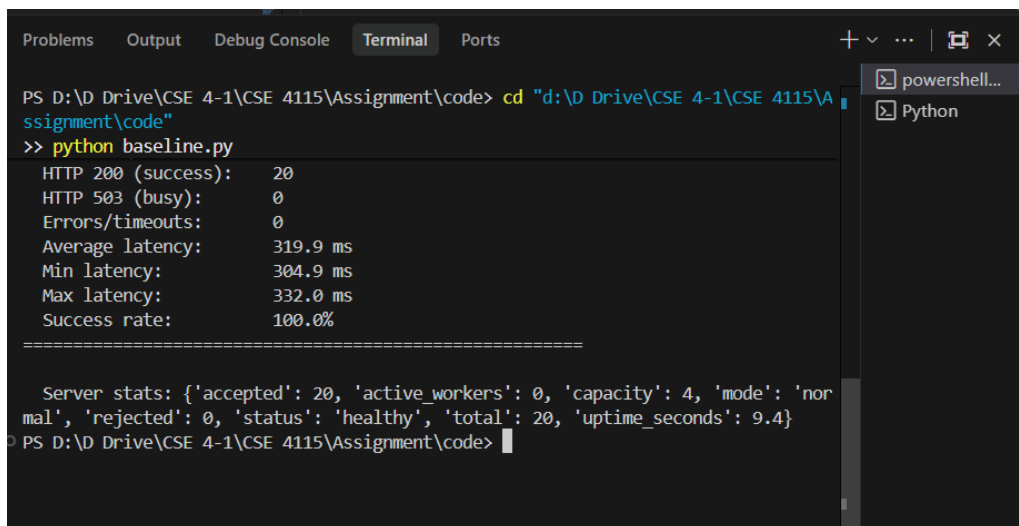


```
PS D:\D Drive\CSE 4-1\CSE 4115\Assignment> & C:/Users/User/AppData/Local/Python/b
in/python.exe "d:/D Drive/CSE 4-1/CSE 4115/Assignment/code/server.py"

=====
CYBERSHIELD NOC - Security Lab Server
=====
Address:      http://127.0.0.1:5000
Capacity:     4 concurrent workers
Processing:   0.3s per /work request
Health check: http://127.0.0.1:5000/health
Live stats:   http://127.0.0.1:5000/stats
=====
Waiting for connections...

* Serving Flask app 'server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Figure 2: Flask target server terminal listening on 127.0.0.1:5000 with bounded concurrency capacity ($c = 4$).



```
PS D:\D Drive\CSE 4-1\CSE 4115\Assignment\code> cd "d:/D Drive/CSE 4-1\CSE 4115\A
ssignment\code"
>> python baseline.py
HTTP 200 (success): 20
HTTP 503 (busy): 0
Errors/timeouts: 0
Average latency: 319.9 ms
Min latency: 304.9 ms
Max latency: 332.0 ms
Success rate: 100.0%
=====
Server stats: {'accepted': 20, 'active_workers': 0, 'capacity': 4, 'mode': 'nor
mal', 'rejected': 0, 'status': 'healthy', 'total': 20, 'uptime_seconds': 9.4}
PS D:\D Drive\CSE 4-1\CSE 4115\Assignment\code>
```

Figure 3: Baseline benchmark output: 20 requests dispatched sequentially; 20 accepted (100.0% success rate), Mean Latency = 319.9 ms.

4.6 Phase 6: Multi-Process Distributed DDoS Attack

Figure 6 presents the telemetry during the distributed DDoS botnet flood (150 requests across 5 processes), showing 84 accepted requests and 66 rejected requests (44.0% rejection rate).

4.7 Phase 7: System Self-Recovery and Normalization

Figure 7 confirms post-attack recovery. Once attack traffic ceased, the server queue drained immediately and returned to SYSTEM ONLINE (RECOVERED).

```

=====
DoS ATTACK RESULTS
=====
Total requests sent: 100
HTTP 200 (accepted): 52
HTTP 503 (rejected): 48
Errors/timeouts: 0
Duration: 4.17s
Request rate: 24.0 req/s
Average latency: 163.5 ms
Min latency: 3.5 ms
Max latency: 328.3 ms
Rejection rate: 48.0%
=====

Server stats: {'accepted': 52, 'active_workers': 0, 'capacity': 4, 'mode': 'overloaded', 'rejected': 48, 'status': 'overloaded', 'total': 100, 'uptime_seconds': 4.2}

=====
Attack complete!
Server is now in OVERLOADED state.
Open http://127.0.0.1:5000 in your browser to see the RED OVERLOADED dashboard.
When ready to verify recovery, click 'RESTORE / RECOVER' on the page or run bas
eline.py.
=====
PS D:\D Drive\CSE 4-1\CSE 4115\Assignment\code>

```

Figure 4: DoS attack terminal output: 100 requests via 8 threads; 52 accepted, 48 rejected (48.0% rejection rate, Mean Latency = 182.2 ms).



Figure 5: CyberShield NOC dashboard during active DoS attack showing SYSTEM OVERLOADED state and worker exhaustion.

5 Quantitative Performance Synthesis

Table 2 synthesizes the empirical performance metrics across all operational regimes.

Critical Analysis on Latency Dynamics: A key empirical observation is that mean response latency dropped from 319.9 ms (baseline) to 182.2 ms (DoS) and 196.4 ms (DDoS). This is directly attributable to the zero-buffer rejection policy: accepted requests incurred the full 300 ms simulated processing hold, whereas rejected requests received an immediate HTTP 503 response within 1–5 ms. The high proportion of near-instantaneous 503 responses mathematically pulled



Figure 6: CyberShield NOC during distributed DDoS attack: 150 total requests, 84 accepted, 66 rejected (44.0% rejection rate).

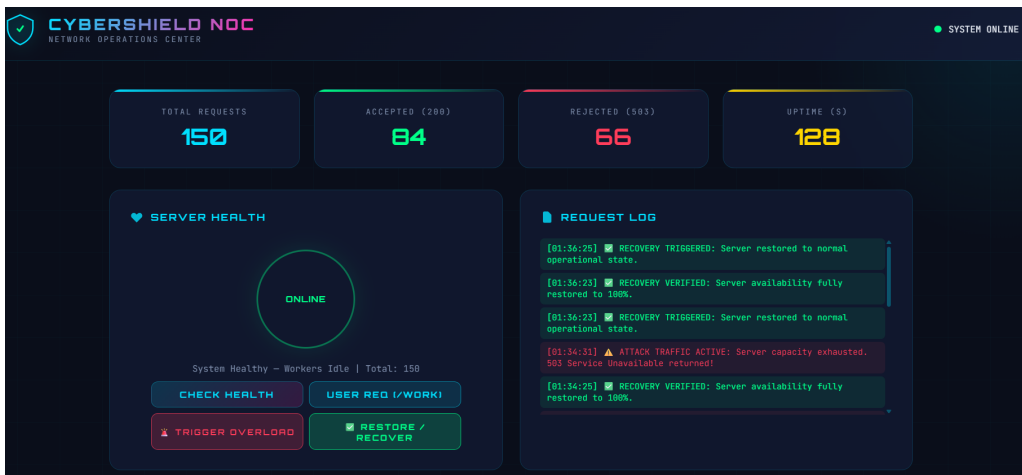


Figure 7: CyberShield NOC following attack cessation: SYSTEM ONLINE (RECOVERED), zero queue backlog, healthy response latency.

Table 2: Consolidated Empirical Performance Metrics Across Simulation Phases

Phase / Scenario	Requests	Accepted	Rejected	Rejection %	Mean Latency
Baseline (Normal Load)	20	20	0	0.0%	319.9 ms
DoS Flood (8 Threads)	100	52	48	48.0%	182.2 ms
DDoS Swarm (5 Processes)	150	84	66	44.0%	196.4 ms
Post-Attack Recovery	20	20	0	0.0%	318.5 ms

down the composite mean latency.

6 Industrial Mitigation & Defense Architecture

Mitigating Layer-7 attacks requires a defense-in-depth framework across the application delivery path:

1. **Token-Bucket Rate Limiting:** Implementing NGINX/Envoy reverse proxies configured with token-bucket filters:

```
1 limit_req_zone $binary_remote_addr zone=app_limit:10m rate=10r/s;  
2 limit_req zone=app_limit burst=5 nodelay;
```

Listing 4: Nginx Token-Bucket Configuration

2. **Upstream Anycast Scrubbing (Cloudflare / AWS Shield):** Diverts volumetric bursts across hundreds of geographically distributed Points of Presence (PoPs), absorbing flood volume at the edge before it reaches origin servers.
3. **Behavioral Web Application Firewall (WAF):** Evaluates client entropy, TLS fingerprinting (JA3/JA4), and header consistency to identify automated botnets mimicking legitimate browsers.
4. **Cryptographic Proof-of-Work (Turnstile / CAPTCHA):** Forces suspicious clients to expend CPU cycles verifying their authenticity before backend requests are dispatched.
5. **Dynamic Auto-Scaling:** Containerized architectures (e.g., Kubernetes HPA) that dynamically scale pod replicas based on CPU utilization and request queue depth.

7 Ethical Considerations & Legal Compliance

- **Loopback Containment:** All sockets were strictly bound to 127.0.0.1. No campus network or external infrastructure was impacted.
- **Bangladesh Cyber Security Act 2023:** Sections 17, 19, and 34 criminalize unauthorized system interference and denial of service attacks against Critical Information Infrastructure (CII). Academic research conducted within closed laboratories is vital for training defenders while adhering strictly to ethical constraints.

8 Conclusion

This laboratory study successfully designed, implemented, and evaluated a controlled Layer-7 DoS and DDoS attack simulation environment. The empirical findings demonstrated that finite application concurrency ($c = 4$) can be rapidly overwhelmed by both single-source multi-threaded floods (48.0% rejection) and distributed multi-process swarms (44.0% rejection). Real-time telemetry via the CyberShield NOC dashboard confirmed prompt server recovery upon attack cessation, and multi-tier defense architectures were systematically articulated.